

👉 TOPIC 03.0: TWO POINTERS  
👉 FOUNDATION & MENTAL MODEL

REUSABLE TWO POINTERS UTILITIES

Element Swapping & Reversal:

```
void swap(int[] arr, int i, int j) {  
    int temp = arr[i]; arr[i] =  
    arr[j]; arr[j] = temp;  
}  
void reverse(int[] arr, int left,  
int right) {  
    while (left < right)  
        swap(arr, left++, right--);  
}
```

Array Partition Utility (Lomuto):

```
int partition(int[] arr, int low,  
int high) {  
    int pivot = arr[high], i =  
    low;  
    for (int j = low; j < high;  
j++) {  
        if (arr[j] ≤ pivot)  
            swap(arr, i++, j);  
    }  
    swap(arr, i, high);  
    return i;  
}
```

PAGE 1 OF 12

💡 AHA MOMENT #1: WHY TWO POINTERS WORKS

Instead of checking all  $O(N^2)$  pairs using nested loops, Two Pointers leverages array properties (like sorting or monotonicity) to prune entire invalid search spaces at every single step, cutting execution time to linear  $O(N)$ !

1 TWO POINTERS FOUNDATION, MECHANICS & BOUNDARIES

What is Two Pointers?

An algorithmic technique using two integer index variables traversing an array or string simultaneously to eliminate nested loop iterations.

When to Use:

- Sorted Arrays (Pair Sums)
- String Palindromes
- In-Place Element Overwriting
- Container Area Bounding

When NOT to Use:

- Unsorted arrays where original indices matter.
- Dynamic variable-length target windows (use Sliding Window instead).

🌳 TWO POINTER PATTERN SELECTION DECISION TREE

STEP 1 Is array sorted OR checking palindrome/container bounds?

↳ YES: Use OPPOSITE DIRECTION POINTERS ( $L = 0, R = N - 1$ )

STEP 2 Is problem asking for in-place overwriting / filtering?

↳ YES: Use SAME DIRECTION POINTERS (Reader / Writer)

STEP 3 Is problem partitioning 3 distinct values (e.g. 0, 1, 2)?

↳ YES: Use DUTCH NATIONAL FLAG (Low, Mid, High)

★ KEY TAKEAWAY: Two Pointers converts  $O(N^2)$  brute-force pair searching into  $O(N)$  linear scans by pruning invalid search space!

💡 AHA MOMENT #2: IS SORTING ALWAYS REQUIRED FOR OPPOSITE POINTERS?

- **FOR PAIR SUMS:** YES! Sorting is strictly mandatory so `L++` guarantees sum increases and `R--` guarantees sum decreases.
- **FOR PALINDROMES & WATER CONTAINERS:** NO! Palindromes check symmetric character equality from outer bounds inward, and Container Water uses structural height bounds!

1 OPPOSITE DIRECTION CORE CONCEPT

Pointers start at opposite ends: `left = 0` and `right = nums.length - 1`. The loop runs while `left < right`. One pointer moves inward per iteration based on comparison logic.

2 OPPOSITE DIRECTION JAVA TEMPLATE

```
public void oppositePointersTemplate(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) {
            left++; right--; // Match found
        } else if (sum < target) {
            left++; // Need larger sum
        } else {
            right--; // Need smaller sum
        }
    }
}
```

🧩 STEP-BY-STEP POINTER MOVEMENT VISUALIZATION: PAIR SUM

Input Array: `nums = [1, 2, 3, 4, 6]`, Target = `6`

STEP 1: Initial Bounds

`L = 0` (val 1), `R = 4` (val 6) | Sum = `1 + 6 = 7` (`7 > 6`) | Action: **R--**

STEP 2: Shrink Right Bound

`L = 0` (val 1), `R = 3` (val 4) | Sum = `1 + 4 = 5` (`5 < 6`) | Action: **L++**

STEP 3: Advance Left Bound

`L = 1` (val 2), `R = 3` (val 4) | Sum = `2 + 4 = 6` | **MATCH FOUND! Return [1, 3]**

⚠️ **COMMON POINTER BUG:** Using `left ≤ right` in pair sum algorithms causes an element to pair with itself! Use `left < right`.

## 💡 AHA MOMENT #3: IN-PLACE MEMORY OVERWRITING

Same Direction Two Pointers acts like a memory filter! The `reader` pointer scans raw unverified input data, while the `writer` pointer maintains the boundary of processed valid output elements in  $O(1)$  extra space.

## 1 SAME DIRECTION CORE CONCEPT

Both pointers start at index 0 and move right. `reader` inspects every element, while `writer` updates only when a valid element is found.

## 2 SAME DIRECTION JAVA TEMPLATE

```
public int sameDirectionTemplate(int[] nums, int val) {
    int writer = 0;
    for (int reader = 0; reader < nums.length; reader++) {
        if (nums[reader] != val) {
            nums[writer] = nums[reader];
            writer++; // Advance output length
        }
    }
    return writer;
}
```

## 🔧 STEP-BY-STEP POINTER MOVEMENT: REMOVE ELEMENT

Input Array: `nums = [3, 2, 2, 3]`, `val = 3`

**STEP 1: `reader = 0` (`val 3 = target val 3`)**

Action: Skip write, advance reader → `W=0`, `R=1`

**STEP 2: `reader = 1` (`val 2 ≠ target val 3`)**

Action: Copy `nums[1]` to `nums[W=0]`, `W++` → Array: `[2, 2, 2, 3]`, `W=1`, `R=2`

**STEP 3: `reader = 2` (`val 2 ≠ target val 3`)**

Action: Copy `nums[2]` to `nums[W=1]`, `W++` → Array: `[2, 2, 2, 3]`, `W=2`, `R=3`

🚀 **KEY TAKEAWAY:** Same Direction Two Pointers allows removing elements or filtering duplicates in  $O(N)$  time with ZERO extra memory allocation!



## 1 MULTIPLE POINTER PATTERN (DUTCH NATIONAL FLAG)

### Three Pointer Region Bounding:

- `low`: Tracks boundary of 0s.
- `mid`: Current scanner pointer.
- `high`: Tracks boundary of 2s.

Classifies 3 regions in a single pass  $O(N)$  time!

```
// DUTCH NATIONAL FLAG TEMPLATE
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid ≤ high) {
        if (nums[mid] == 0) swap(nums, low++, mid++);
        else if (nums[mid] == 1) mid++;
        else swap(nums, mid, high--);
    }
}
```

## 2 PARTITION PATTERN (QUICKSORT & PIVOT)

### Pivot Partitioning Mechanics:

Selects a pivot element and partitions array into elements  $\leq$  pivot on left and elements  $>$  pivot on right. Foundation for QuickSort and QuickSelect ( $O(N)$  average K-th largest selection)!

```
public int partition(int[] nums, int low, int high) {
    int pivot = nums[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (nums[j] ≤ pivot) swap(nums, ++i, j);
    }
    swap(nums, i + 1, high);
    return i + 1;
}
```

## 🔧 JAVA TWO POINTER TOOLKIT & UTILITY HACKS

### In-Place Element Swap Helper:

```
private void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
```

### Duplicate Skipping Loop Trick:

```
// Skip duplicates in 3Sum to avoid duplicate triplets
while (l < r && nums[l] == nums[l + 1]) l++;
while (l < r && nums[r] == nums[r - 1]) r--;
l++; r--;
```

💡 **AHA MOMENT:** Three Pointers (Dutch National Flag) partitions 3 distinct regions in one pass without sorting overhead!


**RECOGNITION SIGNALS: KEYWORD TO POINTER PATTERN MAPPING**

| Problem Trigger Keyword         | Two Pointer Pattern Choice       | Pointer Initialization                      | Target Complexity                                  |
|---------------------------------|----------------------------------|---------------------------------------------|----------------------------------------------------|
| "Sorted Array Target Pair Sum"  | Opposite Direction               | <code>left = 0, right = N - 1</code>        | Time: <code>O(N)</code> , Space: <code>O(1)</code> |
| "Check Palindrome String"       | Opposite Direction               | <code>left = 0, right = N - 1</code>        | Time: <code>O(N)</code> , Space: <code>O(1)</code> |
| "In-Place Remove / Overwrite"   | Same Direction (Reader / Writer) | <code>writer = 0, reader = 0</code>         | Time: <code>O(N)</code> , Space: <code>O(1)</code> |
| "Move Zeros / Compact Array"    | Same Direction (Reader / Writer) | <code>writer = 0, reader = 0</code>         | Time: <code>O(N)</code> , Space: <code>O(1)</code> |
| "Sort Colors / 3-Way Partition" | Multiple Pointers (Dutch Flag)   | <code>low = 0, mid = 0, high = N - 1</code> | Time: <code>O(N)</code> , Space: <code>O(1)</code> |
| "Container Max Area Bounding"   | Opposite Direction Shrinking     | <code>left = 0, right = N - 1</code>        | Time: <code>O(N)</code> , Space: <code>O(1)</code> |


**TWO POINTER ARCHITECTURAL DECISION MATRIX**

**Opposite Direction**

Use when array is sorted or checking symmetric bounds from ends inward.

**Same Direction**

Use for in-place overwriting, zero shifting, or string compression.

**Three Pointers**

Use for 3-way partitioning (Dutch National Flag) or 3Sum anchors.


**FAANG INTERVIEW PRINCIPLE:** "State your pointer orientation strategy to the interviewer BEFORE writing code to demonstrate clarity!"

1. LEETCODE 125 — VALID PALINDROME

Difficulty: EasyPattern: Opposite DirectionMeta, Amazon★★★★★

Problem:

Return

true

if string

s

is a palindrome considering alphanumeric chars only.

Example:

s = "A man, a plan, a canal: Panama"

→

true

.

STEP-BY-STEP POINTER CLEANING & MATCHING

1. left skips non-alphanumeric symbols from left.

2. right skips non-alphanumeric symbols from right.

3. Compare toLowerCase(s.charAt(left)) == toLowerCase(s.charAt(right)).

Complexity:

Time:

O(N)

, Space:

O(1)

.

Why Not Regex Replace?

Creating a new cleaned string takes O(N) extra space. Two Pointers cleans in O(1) space!

```

public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        // Step 1: Skip non-alphanumeric chars from left
        while (left < right &&
!Character.isLetterOrDigit(s.charAt(left))) left++;
        // Step 2: Skip non-alphanumeric chars from right
        while (left < right &&
!Character.isLetterOrDigit(s.charAt(right))) right--;
        // Step 3: Case-insensitive comparison
        if (Character.toLowerCase(s.charAt(left)) !=
Character.toLowerCase(s.charAt(right))) {
            return false;
        }
        left++; right--;
    }
    return true;
}
    
```

2. LEETCODE 167 — TWO SUM II (SORTED ARRAY)

Difficulty: EasyPattern: Opposite DirectionAmazon, Google★★★★★

Problem:

Return 1-based indices of two numbers in sorted array adding to

target

.

Example:

numbers = [2,7,11,15], target = 9

→

[1, 2]

.

```

public int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return new int[]{left + 1, right + 1};
        // 1-based
        else if (sum < target) left++; // Need larger sum
        else right--; // Need smaller sum
    }
    return new int[0];
}
    
```



## 3. LEETCODE 26 — REMOVE DUPLICATES FROM SORTED ARRAY

EasySame DirectionGoogle, Meta★★★★★

**Problem:**

Remove duplicates in-place from sorted

nums

. Return

k

*Example:*

nums = [1, 1, 2]

→

k = 2, nums = [1, 2, \_]

**Complexity:** Time:  $O(N)$ , Space:  $O(1)$ .

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int k = 1; // Writer pointer
    for (int i = 1; i < nums.length; i++) { // Reader
        if (nums[i] != nums[i - 1]) nums[k++] = nums[i];
    }
    return k;
}
```

## 4. LEETCODE 27 — REMOVE ELEMENT

EasySame DirectionAmazon, Microsoft★★★★★

**Problem:**

Remove occurrences of

val

in

nums

in-place. Return

k

*Example:*

nums = [3, 2, 2, 3], val = 3

→

k = 2

**Complexity:** Time:  $O(N)$ , Space:  $O(1)$ .

```
public int removeElement(int[] nums, int val) {
    int k = 0; // Writer
    for (int i = 0; i < nums.length; i++) { // Reader
        if (nums[i] != val) nums[k++] = nums[i];
    }
    return k;
}
```

## 5. LEETCODE 283 — MOVE ZEROES

EasySame DirectionSwapFacebook, Apple★★★★★

**Problem:**

Move all 0's to the end of

nums

while preserving relative order

```
public void moveZeroes(int[] nums) {
    int write = 0;
    for (int read = 0; read < nums.length; read++) {
        if (nums[read] != 0) {
            int temp = nums[write];
            nums[write] = nums[read];
            nums[read] = temp;
        }
    }
}
```

**6. LEETCODE 88 — MERGE SORTED ARRAY (BACKWARDS FILL)****Difficulty:** Easy **Pattern:** Backwards Three Pointers **Meta, Amazon, MS** ★★★★★**Problem:**

Merge sorted

nums2

into sorted

nums1

in-place.

*Example:*

nums1 = [1,2,3,0,0,0], m=3, nums2 = [2,5,6], n=3

→

[1,2,2,3,5,6]

.

**Key Insight:**

Merge from index

m+n-1

backwards!

```
public void merge(int[] nums1, int m, int[] nums2, int n) {
    int p1 = m - 1, p2 = n - 1, p = m + n - 1;
    while (p2 ≥ 0) {
        if (p1 ≥ 0 && nums1[p1] > nums2[p2]) nums1[p--] =
            nums1[p1--];
        else nums1[p--] = nums2[p2--];
    }
}
```

**7. LEETCODE 443 — STRING COMPRESSION****Difficulty:** Medium **Pattern:** Reader / Writer **Amazon, Microsoft** ★★★★★**Problem:**

Compress character array in-place using consecutive counts.

*Example:*

chars = ["a","a","b","b","c","c","c"]

→

["a","2","b","2","c","3"]

, return

6

.

```
public int compress(char[] chars) {
    int write = 0, read = 0;
    while (read < chars.length) {
        char currentChar = chars[read];
        int count = 0;
        while (read < chars.length && chars[read] == currentChar) {
            read++; count++;
        }
        chars[write++] = currentChar;
        if (count > 1) {
            for (char c : String.valueOf(count).toCharArray()) {
                chars[write++] = c;
            }
        }
    }
    return write;
}
```

💡 **AHA MOMENT (LC 443):** Using inner while loop inside Reader/Writer advances `read` through contiguous duplicate blocks cleanly!





## 8. LEETCODE 15 — 3SUM (SORT + ANCHOR + TWO POINTERS)

**Difficulty:** Medium **Pattern:** Anchor + Opposite Pointers **Meta, Amazon, Google** ★★★★★**Problem:**

Return all unique triplets

`[nums[i], nums[j], nums[k]]`

such that sum is

`0`

.

*Example:*`nums = [-1, 0, 1, 2, -1, -4]`

→

`[[-1,-1,2], [-1,0,1]]`

.

**Evolution Strategy:**

- Sort array.
- Fix anchor `i` from `0` to `N-3`.
- Run Two Pointers (`left = i+1, right = N-1`) for target `-nums[i]`!

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // Skip dup
        anchor
        int l = i + 1, r = nums.length - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                while (l < r && nums[l] == nums[l + 1]) l++; //
                while (l < r && nums[r] == nums[r - 1]) r--; //
                l++; r--;
            } else if (sum < 0) l++; else r--;
        }
    }
    return res;
}
```

## 9. LEETCODE 75 — SORT COLORS (DUTCH NATIONAL FLAG)

**Difficulty:** Medium **Pattern:** Dutch National Flag **Meta, Amazon, MS** ★★★★★**Problem:**

Sort array with 0s, 1s, and 2s in-place in a single pass.

*Example:*`nums = [2,0,2,1,1,0]`

→

`[0,0,1,1,2,2]`

.

```
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid ≤ high) {
        if (nums[mid] == 0) {
            swap(nums, low++, mid++);
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            swap(nums, mid, high--);
        }
    }
}

private void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
```

**AHA MOMENT (LC 15):** Sorting reduces 3Sum from  $O(N^3)$  to  $O(N^2)$  and allows duplicate skipping via simple pointer checks!

## 10. LEETCODE 11 — CONTAINER WITH MOST WATER

Medium/HardArea Bounds ShrinkingMeta, Amazon, Google★★★★★

### Problem:

Find two lines that together with x-axis form container holding max water.

Example:

height = [1,8,6,2,5,4,8,3,7]

→

49

### AREA SHRINKING LOGIC

Always move the SHORTER line pointer inward because moving the taller line can NEVER increase area!

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1, maxWater = 0;
    while (left < right) {
        int w = right - left;
        int h = Math.min(height[left], height[right]);
        maxWater = Math.max(maxWater, w * h);
        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}
```

## 11. LEETCODE 42 — TRAPPING RAIN WATER (TWO POINTER OPTIMAL)

Difficulty: HardTwo Pointer BoundsGoogle, Meta, Amazon★★★★★

### Problem:

Compute total water trapped given elevation map

height

.

Example:

height = [0,1,0,2,1,0,1,3,2,1,2,1]

→

6

.

### O(1) Space Improvement:

Replaces  $O(N)$  prefix/suffix arrays with dynamic two-pointer max bounds!

```
public int trap(int[] height) {
    int l = 0, r = height.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            if (height[l] ≥ leftMax) leftMax = height[l];
            else water += leftMax - height[l];
            l++;
        } else {
            if (height[r] ≥ rightMax) rightMax = height[r];
            else water += rightMax - height[r];
            r--;
        }
    }
    return water;
}
```

 **MEMORY BOX:** Moving the shorter height pointer in Container Water guarantees we never skip a potentially larger water container!

⚠️ TOP 10 COMMON TWO POINTER BUGS & FAANG TRAPS

1. **Loop Termination Crash:** Using `while(l ≤ r)` instead of `while(l < r)` in pair sum algorithms.

2. **Missing Boundary Checks in Duplicate Skips:** Forgetting `l < r` in inner duplicate loops causing `IndexOutOfBoundsException`.

3. **Applying Two Pointers on Unsorted Inputs:** Running Opposite Pointers on unsorted arrays for pair sum!

4. **Forgetting to Advance Both Pointers on Match:** Advancing only one pointer after finding a valid 3Sum triplet.

5. **Modifying Read Pointer Incorrectly:** Modifying `read` pointer inside `write` logic.
6. **Off-By-One In-Place Output Length:** Returning `write + 1` instead of `write`.

7. **Integer Overflow in Area Math:** Failing to cast width/height multiplication in large arrays.

8. **Moving Taller Pointer in Water Container:** Violating area optimization logic.

9. **Inappropriate Pointer Direction:** Using Same Direction for Palindrome checks.

10. **Skipping Duplicate Anchor in 3Sum:** Forgetting `'i > 0 && nums[i] == nums[i-1]'` duplicate check.

🔍 REVERSE QUESTIONS DECISION TREE ("WHY NOT?")

| Alternative Choice                    | Why We Reject It In FAANG Interviews                                                                |
|---------------------------------------|-----------------------------------------------------------------------------------------------------|
| Why Not Brute Force $O(N^2)$ ?        | Nested loops exceed time limits ( $N \geq 10^5$ ). Two Pointers prunes search space to $O(N)!$      |
| Why Not HashMap $O(N)$ Space?         | When array is ALREADY sorted, Two Pointers achieves $O(1)$ auxiliary space complexity.              |
| Why Not Binary Search $O(N \log N)$ ? | Binary search takes $O(N \log N)$ total time for $N$ queries vs $O(N)$ total time for Two Pointers. |

💡 **FAANG COACH TIP:** Mentioning why Two Pointers eliminates HashMap  $O(N)$  space overhead proves your memory optimization mastery!

★ TWO POINTER MASTER PATTERN MATRIX

| Pattern Variant             | Pointer Setup                                       | Optimal Complexity                                  | Representative LeetCode Problem         |
|-----------------------------|-----------------------------------------------------|-----------------------------------------------------|-----------------------------------------|
| Opposite Direction          | <code>L = 0, R = N - 1</code>                       | Time: <code>O(N)</code> , Space: <code>O(1)</code>  | LC 125 Palindrome, LC 167 Two Sum II    |
| Same Direction              | <code>writer = 0, reader = 0</code>                 | Time: <code>O(N)</code> , Space: <code>O(1)</code>  | LC 26 Remove Dupes, LC 283 Move Zeroes  |
| Fixed Anchor + Two Pointers | <code>i</code> loop + <code>L = i+1, R = N-1</code> | Time: <code>O(N²)</code> , Space: <code>O(1)</code> | LC 15 3Sum                              |
| Dutch National Flag         | <code>low = 0, mid = 0, high = N-1</code>           | Time: <code>O(N)</code> , Space: <code>O(1)</code>  | LC 75 Sort Colors                       |
| Two Pointer Area Bounds     | <code>L = 0, R = N - 1</code>                       | Time: <code>O(N)</code> , Space: <code>O(1)</code>  | LC 11 Container Water, LC 42 Rain Water |

 NEETCODE & BLIND 75 PROGRESS TRACKER

- ☒ LC 125 Valid Palindrome
- ☒ LC 167 Two Sum II
- ☒ LC 26 Remove Duplicates
- ☒ LC 27 Remove Element
- ☒ LC 283 Move Zeroes
- ☒ LC 88 Merge Sorted Array
- ☒ LC 15 Three Sum
- ☒ LC 75 Sort Colors
- ☒ LC 443 String Compression
- ☒ LC 11 Container Water
- ☒ LC 42 Trapping Rain Water

✓ CHECKLIST BEFORE MOVING TO TOPIC 04 (SLIDING WINDOW)

- ☒ I understand Opposite Direction vs Same Direction vs Dutch National Flag patterns.
- ☒ I master duplicate skipping loops in 3Sum and in-place Reader/Writer overwriting.
- ☒ I can write Two Sum II, 3Sum, Sort Colors & Container Water from memory in <3 mins.

 TOPIC 03 TWO POINTERS COMPLETE! Turn the page to **Topic 04: Sliding Window Masterclass!**